

Ask Campus

RAG Chatbot Challenge

Career Fair • 2-Day Challenge

Duration	2 Days
Difficulty	Beginner → Intermediate
AI Allowed	Yes — use any LLM API, framework, or AI tool you like
Deliverable	A working chatbot that answers questions about school records

Overview

You are given a CSV file containing university student records — enrollments, classes, instructors, scores, and grades.

Your task is to build a chatbot that lets anyone ask natural language questions about that data and get accurate, grounded answers.

This is a classic Retrieval-Augmented Generation (RAG) challenge: the model should answer based on the data you give it, not from its own training knowledge.

Sample Questions Your Chatbot Should Handle

- What classes is Ahmed enrolled in?
- How many students are in CS101?
- Who scored the highest in Mathematics?
- What is the average score across all classes?
- Which class has the most students?
- Show me all students who received an A grade.

Challenge Levels

Each level builds on the previous one. You must complete a level before moving to the next level.

LEVEL 1 - Get the Data Talking

Requirements

- Load and parse the provided CSV file
- Accept a plain text question from the user (CLI or notebook is fine)
- Pass the question + CSV content to an LLM API
- Return a readable, accurate answer
- Handle at least: enrollment lookup, class headcount, highest score queries

Technical Hints

- *No need for embeddings yet — just send the full CSV as context*
- *Try OpenAI, Gemini, Groq, or any free-tier LLM API*
- *Prompt engineer carefully: tell the model to only answer from the data*
- *Watch out for hallucinations — the model should say 'not found' if the data doesn't have it*
- *pandas is your friend for pre-processing before sending to the LLM*

LEVEL 2 - Go Deeper

Requirements

- Handle aggregation queries: averages, rankings, comparisons across classes
- Handle follow-up questions with context (e.g., 'what about that class?' after asking about Math)
- Return structured answers where appropriate (e.g. a ranked list of top scorers)
- The bot should never confidently answer something not in the data

Technical Hints

- *Maintain a simple conversation history array and pass it in each API call*
- *Consider pre-computing aggregates with pandas and injecting them as a summary*
- *For rankings, let the code do the sorting don't trust the LLM to sort numbers correctly*
- *Test edge cases: tied scores, dropped students, missing grades*
- *Add a system prompt that defines the bot's role and constraints clearly*

LEVEL 3 - Make It Usable

Requirements

- Build a simple chat UI — web page, or desktop app
- Support back-and-forth conversation with visible chat history
- Show a clear indicator when the bot is thinking / loading
- Handle graceful errors: API failures, bad input, data not found
- Bonus: let the user upload their own CSV and chat with it

Technical Hints

- *A simple HTML + JS page or a Streamlit/Gradio app works perfectly*
- *Store conversation state client-side or in a local variable*
- *Debounce the send button so users can't double-submit*
- *Display the source row or evidence alongside the answer for transparency*
- *For the bonus: use a file input, parse it in-browser or send it to your backend*

Judging Criteria

Maximum score: 100 points. Judges will test your bot live with questions from the CSV.

Criteria	What We Look For	Points
Accuracy	Answers are correct and grounded in the CSV data	25 pts
Hallucination Control	Bot does not make up data or answer outside its scope	20 pts
Level Completion	How many levels were fully completed	15 pts
Conversation Quality	Follow-up questions work, context is maintained	15 pts
Code Quality	Clean, readable, well-structured code, NO HARD CODED API KEYS	15 pts
UI / UX (Level 3)	Interface is intuitive and handles errors gracefully	10 pts

- Extra **20** points would be given for students who are able to containerize the app and provide a good high level readme file on how to setup and start. If you decide to host and deploy that would be an extra **10** points
- There is no need to provide an API key.

Tips for Success

- Prompt engineering matters more than the model you choose.
- Pre-process your CSV with code for aggregations; don't rely on the LLM for math.
- Test with tricky questions early: tied scores, students in multiple classes, empty fields.
- A bot that says 'I don't know' accurately is better than one that confidently lies.
- Use AI tools freely; the challenge is about what you build, not how you build it.